

Instruction Name	Type	Explanation	Executed On	Pseudo-code	PC	Where is it used
add.s	R-Int	Addition	Scalar Core	rd = rs1 + rs2	PC = PC + 20	Loop Indexing
sub.s	R-Int	Subtraction	Scalar Core	rd = rs1 - rs2	PC = PC + 20	Loop Indexing
mul.s	R-Int	Multiplication	Scalar Core	rd = rs1 * rs2	PC = PC + 20	Loop Indexing
div.s	R-Int	Division	Scalar Core	rd = rs1 / rs2	PC = PC + 20	Count Number of Tiles
mod.s	R-Int	Modulous	Scalar Core	rd = rs1 % rs2	PC = PC + 20	Non-powers of 2 tiles
or.s	R-Int	Bitwise Or	Scalar Core	rd = rs1 rs2	PC = PC + 20	Assemble RS3 for the SDMA and other instructions
and.s	R-Int	Bitwise And	Scalar Core	rd = rs1 & rs2	PC = PC + 20	Assemble RS3 for the SDMA and other instructions
xor.s	R-Int	Bitwise Xor	Scalar Core	rd = rs1 ^ rs2	PC = PC + 20	Assemble RS3 for the SDMA and other instructions
sll.s	R-Int	Shift left logical	Scalar Core	rd = rs1 << rs2	PC = PC + 20	
srl.s	R-Int	Shift Right Logical	Scalar Core	rd = rs1 >> rs2	PC = PC + 20	
sra.s	R-Int	Shift Right Arith	Scalar Core	rd = rs1 >> rs2	PC = PC + 20	
slt.s	R-Int	Set Less Than	Scalar Core	rd = rs1 < rs2	PC = PC + 20	
sltu.s	R-Int	Set Less Than (U)	Scalar Core	rd = u(rs1) < u(rs2)	PC = PC + 20	
bft.s	R-Int	bFloat to scalar	Scalar Core	rd = Int(rs1)	PC = PC + 20	Convert Scalar to BF16, makes compiler's life really easy.
add.bf	R-Float	Add bFloat	Scalar Core	rd = rs1 + rs2	PC = PC + 20	
sub.bf	R-Float	Sub bFloat	Scalar Core	rd = rs1 - rs2	PC = PC + 20	
mul.bf	R-Float	Mult bFloat	Scalar Core	rd = rs1 * rs2	PC = PC + 20	
rec.bf	R-Float	Reciprocal bFloat	Scalar Core	rd = 1 / rs1	PC = PC + 20	all vector divisions in the kernels can be simplified 1/scalar * elementwise multiply.
slt.bf	R-Float	Set Less Than BF16	Scalar Core	rd = rs1 < rs2	PC = PC + 20	
stb.f	R-Float	scalar to bFloat	Scalar Core	rd = Float(rs1)	PC = PC + 20	Convert BF16 to Scalar, makes compiler easier to build.
addi.s	I-Type-Int	Addition Immediate	Scalar Core	rd = rs1 + imm	PC = PC + 20	
subi.s	I-Type-Int	Subtraction Immediate	Scalar Core	rd = rs1 - imm	PC = PC + 20	
muli.s	I-Type-Int	Multiplication Immediate	Scalar Core	rd = rs1 * imm	PC = PC + 20	
divi.s	I-Type-Int	Division Immediate	Scalar Core	rd = rs1 / imm	PC = PC + 20	
modi.s	I-Type-Int	Modulous Immediate	Scalar Core	rd = rs1 % imm	PC = PC + 20	
ori.s	I-Type-Int	Bitwise Or Immediate	Scalar Core	rd = rs1 imm	PC = PC + 20	
andi.s	I-Type-Int	Bitwise And Immediate	Scalar Core	rd = rs1 & imm	PC = PC + 20	
xori.s	I-Type-Int	Bitwise Xor Immediate	Scalar Core	rd = rs1 ^ imm	PC = PC + 20	
slli.s	I-Type-Int	Shift left logical Immediate	Scalar Core	rd = rs1 << (imm & 0x1F)	PC = PC + 20	
srli.s	I-Type-Int	Shift Right Logical Immediate	Scalar Core	rd = rs1 >> (imm & 0x1F)	PC = PC + 20	
sra.i.s	I-Type-Int	Shift Right Arith Immediate	Scalar Core	rd = rs1 >> (imm & 0x1F)	PC = PC + 20	
slti.s	I-Type-Int	Set Less Than Immediate	Scalar Core	rd = rs1 < imm	PC = PC + 20	
sltiu.s	I-Type-Int	Set Less Than (U) Immediate	Scalar Core	rd = u(rs1) < u(imm)	PC = PC + 20	
beq.s	B-type	Branch Equal To (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 == rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.
bne.s	B-type	Branch Not Equal To (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 != rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.
blt.s	B-type	Branch Less Than (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 < rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.
bge.s	B-type	Branch Greater Than or Equal to (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 >= rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.
bgt.s	B-type	Branch Greater Than (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 > rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.

ble.s	B-type	Branch Less Than or Equal to (Int)	Scalar Core	rs1 = rs1 + incr7;	if (rs1 <= rs2): PC = PC + imm else: PC = PC + 20	Loops, any and all of them! The workload has a ton of loops.
lw.s	M-type	Load Word	Scalar Core	rd = MEM[rs1 + sign_ext(imm12)]	PC = PC + 20	Bookkeeping logic.
sw.s	M-type	Store Word	Scalar Core	MEM[rs1 + sign_ext(imm12)] = rd	PC = PC + 20	Bookkeeping logic.
lhw.s	M-type	Load Half-Word	Scalar Core	rd[15:0] = MEM[rs1 + sign_ext(imm12)][15:0]	PC = PC + 20	Handling bf16 scalar reads, makes compiler simple.
shw.s	M-type	Store Half-Word	Scalar Core	MEM[rs1 + sign_ext(imm12)][15:0] = rd[15:0]	PC = PC + 20	Handling bf16 scalar reads, makes compiler simple.
jal	MI-Type	Jump and Link	Scalar Core	rd = PC + 20;	PC += imm	Function Call for Software Libraries
jalr	MI-Type	Jump and Link Return	Scalar Core	rd = PC + 20;	PC = rs1 + imm	Function Call for Software Libraries
li.s	MI-Type	Load Immediate (Pseudo instruction)	Scalar Core	lui and then addi	PC = PC + 20	Assemble RS3 for the SDMA and other instructions
lui.s	MI-Type	Load Upper Immediate	Scalar Core	rd[31:7] = imm[24:0]	PC = PC + 20	Assemble RS3 for the SDMA and other instructions
nop.s	R-Int	No Op	Scalar Core	no op	PC = PC + 20	Padding where needed
halt.s	R-Int	Halt	Scalar Core	halt	PC = PC + 20	Halting is a important constraint!
add.vv	Vector Vector	Elementwise Vector Add	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = vs1[i] + vs2[i];	PC = PC + 20	Partial Sum Accumulation
sub.vv	Vector Vector	Elementwise Vector Subtract	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = vs1[i] - vs2[i];	PC = PC + 20	Couldn't find use, but virtually no extra hardware to support.
mul.vv	Vector Vector	Elementwise Vector Multiply	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = vs1[i] * vs2[i];	PC = PC + 20	Could be used in a sequence to compute the hadamard product. Elementwise multiply here, and then the reduction sum is used.
gemm.vv	Vector Vector	Issue Vector for GeMM on Systolic Array	Vector Core (Executed on Systolic Array).	vd = vs1 * weights_sys_array;	PC = PC + 20	Issue all GeMMs to the systolic array
expi.vi	Vector Immediate	Elementwise e^x	Vector Core	for (i=0; i<VL; i++) if (mask[i]) vd[i] = exp(vs1[i]);	PC = PC + 20	Elementwise Exponential needed for SoftMax
lw.vi	Vector Immediate	Load a vector into systolic array as weights	Vector Core (Executed on Systolic Array).	weights_sys_array.push(vs[1])	PC = PC + 20	Load weights into the systolic array
rsum.vi	Vector Immediate	Reduction: sum	Vector Core	result = SUM(MASKED(vs)) for i = 0 to VL-1: if imm[6]: vd[i] = result else if i == imm[0:4]: vd[i] = result else if imm[5]: vd[i] = 0 else: vd[i] = vs[i]	PC = PC + 20	Reduction for SoftMax
rmin.vi	Vector Immediate	Reduction: min	Vector Core	result = MIN(MASKED(vs)) for i = 0 to VL-1: if imm[6]: vd[i] = result else if i == imm[0:4]: vd[i] = result else if imm[5]: vd[i] = 0 else: vd[i] = vs[i]	PC = PC + 20	Couldn't find use, but no extra hardware to support.
rmax.vi	Vector Immediate	Reduction: max	Vector Core	result = MAX(MASKED(vs)) for i = 0 to VL-1: if imm[6]: vd[i] = result else if i == imm[0:4]: vd[i] = result else if imm[5]: vd[i] = 0 else: vd[i] = vs[i]	PC = PC + 20	MaxPool

mgt.mvv	Vector Mask Vector	Mask Greater Than (Vector Vector)	Vector Core			Top-k choices, it might be useful for that.
mlt.mvv	Vector Mask Vector	Mask Less Than (Vector Vector)	Vector Core			Top-k choices, it might be useful for that.
meq.mvv	Vector Mask Vector	Mask Equal to (Vector Vector)	Vector Core			Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point. Else programs will crash based on errors that may be there.
mneq.mvv	Vector Mask Vector	Mask Not Equal to (Vector Vector)	Vector Core			Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point. Else programs will crash based on errors that may be there.
mgt.mvs	Vector Mask Scalar	Mask Greater Than (Vector Scalar)	Vector Core (Scalar Value Broadcasted)			ReLU, LeakyReLU, odd sized matrices where the size is a scalar.
mlt.mvs	Vector Mask Scalar	Mask Less Than (Vector Scalar)	Vector Core (Scalar Value Broadcasted)			ReLU, LeakyReLU, odd sized matrices where the size is a scalar.
meq.mvs	Vector Mask Scalar	Mask Equal to (Vector Scalar)	Vector Core (Scalar Value Broadcasted)			Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point. Else programs will crash based on errors that may be there. We might want to use this to check for Infinity maybe?
mneq.mvs	Vector Mask Scalar	Mask Not Equal to (Vector Scalar)	Vector Core (Scalar Value Broadcasted)			Actually not that useful, since we wanna subtract and check that the diff is lower than some threshold for floating point.
vreg.ld	Vector Memory	Vector Load from Scratchpad				All programs that use the vector core need to load data to that.
vreg.st	Vector Memory	Vector Store to Scratchpad				All programs that use the vector core need to load data to that.
scpad.ld	SDMA	Scratchpad Load from DRAM				All tensors are loaded to the scpad.
scpad.st	SDMA	Scratchpad Store to DRAM				All tensors are loaded to the scpad.
vmov.vts		Move 1 element of a vector to scalar				Used in SoftMax since division (reciprocal) is only in the scalar core. It has to be moved and then broadcast back.
mv.mts		Move Selected Vector Mask to Scalar Core				Mask based on vector length than other variables.
mv.stm		Move Mask from Scalar core to Vector Mask File				Mask based on vector length than other variables.
add.vs		Add Scalar to all vector elements	Vector Core (Scalar Value Broadcasted)			Couldn't find use, but virtually no extra hardware to support.
sub.vs		Subtract Scalar from all vector elements	Vector Core (Scalar Value Broadcasted)			Couldn't find use, but virtually no extra hardware to support.
mul.vs		Mult Scalar to all vector elements	Vector Core (Scalar Value Broadcasted)			Softmax uses this.